Documentation technique – Easy ACC

1. Base de Données et Sauvegarde Automatique 3
   1.1 Architecture de la Base de Données (SQLAlchemy) 3
   1.2 La Sérialisation de l'État (State Serialization) 3
   1.3 Mécanisme de Synchronisation (Auto-Save) 3
   1.4 Gestion des Datasets et Suppression Interactive 4
2. Génération des Rapports PDF Interactifs 4
   2.1 Extraction et Moteurs Graphiques 4
   2.2 Assemblage Documentaire (FPDF) 4
   2.3 Traitement des Octets et Intégration Streamlit 5
3. Module de configuration du projet 5
   3.1 Section Infos générales 5
   3.2 Contexte de l'Application 6
   3.3 Composants de l'Interface (UI) 6
   3.4 Carte Interactive et Géocodage 7
   3.5 Mécanisme de Sauvegarde Automatique 7
   3.6 Résumé des Clés de l'État de Session 7
4. Module d'insertion des points d'injection 8
   4.1 Introduction et Objectifs 8
   4.2 Architecture et Organisation 8
   4.3 Informations Collectées par Point d'Injection 8
   4.4 Modes d'Acquisition des Courbes de Production 9
   4.5 Traitement et Validation des Données 9
   4.6 Géolocalisation et Contraintes de Distance 9
   4.7 Agrégation et Préparation pour les Calculs 10
   4.8 Intégration avec le Système ACI 10
   4.9 Persistance et Sauvegarde des Données 10
5. Module d'insertion des points de soutirage 10
   5.1 Introduction et Objectifs 10
   5.2 Architecture et Organisation 11
   5.3 Informations Collectées par Point de Soutirage 11
   5.4 Mode d'Acquisition des Courbes de Consommation 11
   5.5 Traitement et Validation des Données 12
   5.6 Géolocalisation et Contraintes de Distance 13

5.7 Agrégation et Préparation pour les Calculs 13
5.8 Intégration avec le Système ACI 13
5.9 Persistance et Sauvegarde des Données 13
6. Module de configuration des clés de répartition 14
6.1 Architecture générale du module 14
6.2 Gestion de l'état de l'application 14
6.3 Mécanisme de relancement de l'application 15
6.4 Sauvegarde automatique des paramètres 15
6.5 Nettoyage des consommateurs 15
6.6 Modes de répartition disponibles 16
6.7 Mode de répartition par défaut 16
6.8 Mode de répartition statique 16
6.9 Mode de répartition dynamique simple 17
6.10 Configuration des groupes de priorité 18
6.11 Navigation entre les étapes (clé dynamique simple) 18
6.12 Validation et enregistrement des paramètres (clé dynamique simple) 19
6.13 Interaction avec le module de calcul énergétique 19
7. Module de calcul et d'analyse du bilan énergétique 19
7.1 Architecture générale du module 19
7.2 Données d'entrée 20
7.3 Calcul des indicateurs globaux 20
7.4 Fonction principale de calcul 21
7.5 Structures de données utilisées dans l'algorithme 21
7.6 Simulation temporelle 22
7.7 Cas sans production 22
7.8 Allocation prioritaire ACI 22
7.9 Calcul de la production restante 23
7.10 Modes de répartition 23
7.11 Calcul du surplus 24
7.12 Calcul de l'autoconsommation des producteurs 25
7.13 Calcul des indicateurs énergétiques 25
7.14 Visualisation des résultats 25

1. Base de Données et Sauvegarde Automatique

1.1 Architecture de la Base de Données (SQLAlchemy)

L'application utilise une base de données relationnelle SQLite, couplée avec l'ORM (Object-Relational Mapping) SQLAlchemy. Cela garantit l'intégrité des opérations (transactions ACID) et facilite le mapping entre les objets Python et les tables de la base.

Deux tables principales structurent les données :

- Project : Représente une session ou un projet (ex: "Projet A"). Héberge un champ JSON global d'état du projet (`state_payload`) contenant toutes les métadonnées (nom, distance critique, dates, etc.) à l'exception des lourds jeux de données.
- Dataset : Les données volumineuses (courbes de production et de consommation) sont extraites du projet et stockées dans une table native. Chaque dataset possède des colonnes comme 'csv_data' (le dataframe brut sérialisé) et des métadonnées (type_point, point_id), pointant vers le 'Project' parent via une clé étrangère avec la clause de cascade ON DELETE CASCADE.

1.2 La Sérialisation de l'État (State Serialization)

L'application Streamlit agit comme une machine à états (State Machine) via son `st.session_state`. Lors de la sauvegarde du projet, l'intégralité du contexte doit être convertie en un format JSON lisible. Cependant, Streamlit stocke des objets complexes (datetime.date, DataFrames pandas, Callbacks).

La fonction `serialize_state()` :

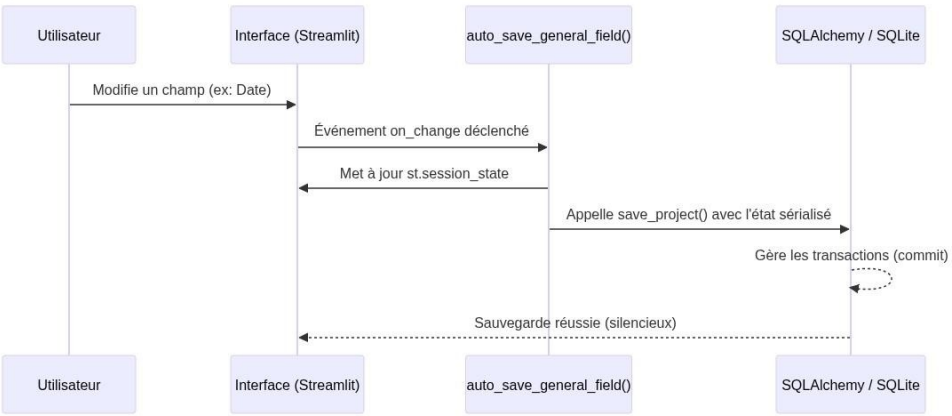
1. Identifie et ignore les clés privées (commençant par des underscores).

2. Extrait et stocke les DataFrames dans la table `Dataset` en arrière-plan, en retirant la donnée binaire lourde du dictionnaire d'état.

3. Convertit les objets natifs (ex: datetime) en chaînes de caractères (ex: '2025-01-01').

1.3 Mécanisme de Synchronisation (Auto-Save)

Pour éliminer le besoin de boutons de sauvegarde manuelle, l'application utilise une architecture événementielle couplée aux widgets Streamlit via le callback `on_change`. Lorsqu'un utilisateur modifie un formulaire, la fonction intermédiaire `auto_save_general_field` intercepte la nouvelle valeur, l'injecte dans le `session_state`, puis appelle dynamiquement la fonction de sauvegarde `services.database.save_project()` de manière asynchrone.



Flux de données (Data flow) de la fonctionnalité Auto-Save

1.4 Gestion des Datasets et Suppression Interactive

Chaque formulaire d'édition de client (Point de soutirage ou Injection) permet des interactions dynamiques : le client conserve son entité dans l'état, mais l'utilisateur a la possibilité de supprimer la courbe attachée et d'en uploader une autre. Toute suppression sur l'interface (via le bouton `Supprimer la courbe`) met immédiatement à jour les clés associées de l'état (ex: 'curve_data' => None), ce qui déclenche une requête sur `save_project` et nettoie automatiquement la table `Dataset` de la base de données. Inversement, chaque validation crée une nouvelle entrée.

2. Génération des Rapports PDF Interactifs

2.1 Extraction et Moteurs Graphiques

Easy ACC utilise Plotly pour l'affichage interactif des données (donuts de répartition, évolution temporelle). Cependant, Plotly est propulsé par JavaScript (navigateur utilisateur). Pour créer un PDF hors-ligne, nous devons convertir ces graphiques dynamiques en images statiques directement depuis le serveur backend Python.

La librairie `kaleido` (*lien documentation :* https://plotly.com/python/static-image-export/#kaleido) agit comme un mini-moteur de navigateur (headless) très rapide. Elle prend les objets `go.Figure` en mémoire et appelle la méthode `fig.write_image()`, calculant tous les pixels et exportant vers des chemins graphiques temporaires partagés sur le serveur (.png).

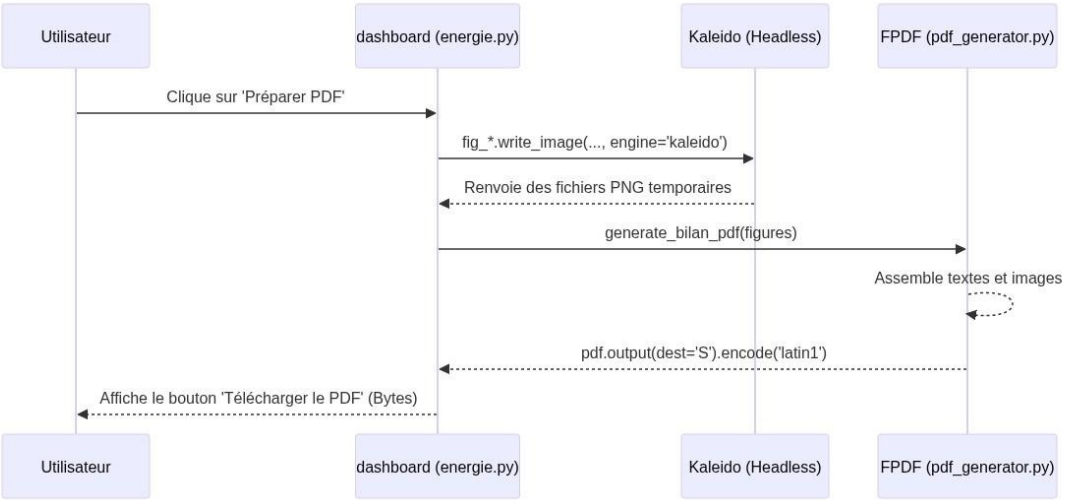
2.2 Assemblage Documentaire (FPDF)

Le cœur de l'exportation réside dans `services/pdf_generator.py`. Pour contourner les limites d'exportation de page unique, la librairie `FPDF` est employée (via la sous-classe personnalisée `BilanPDF`). Le processus est le suivant:

• Initialisation : Un canvas A4 vide est créé avec des en-têtes et pieds de page standardisés.
• Insertion Textuelle : Les métadonnées du projet formatées (Titre, codes postaux, algorithmes utilisés) sont dessinées via `pdf.cell()`.
• Insertion Image : La méthode `pdf.image()` pointe vers les fichiers `.png` extraits par Kaleido et les insère dans le PDF de manière mathématique (gestion manuelle des coordonnées `x` et `y`) pour aligner parfaitement deux rangées de donuts et les graphiques linéaires de bas de document.

2.3 Traitement des Octets et Intégration Streamlit

Une difficulté classique en Python 3 concerne l'encodage des formats binaires comme les `.pdf`. L'appel final `pdf.output(dest='S')` génère le fichier sous forme d'une chaîne de caractères (string) non sécurisée pour le web. Pour que Streamlit puisse lancer le téléchargement (`st.download_button`), nous encodons délibérément la chaîne source en un tableau binaire structuré (.encode('latin1')). Le frontend transmet ensuite dynamiquement ce Blob au navigateur utilisateur via un mime: application/pdf.



Cycle de vie complet de l'extraction et construction du rapport PDF

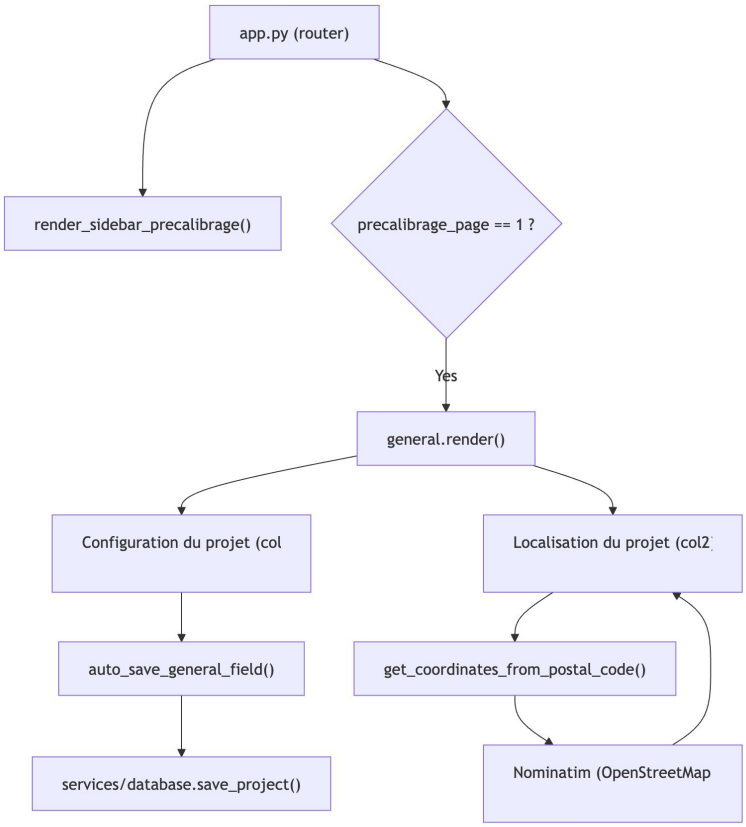
3. Module de configuration du projet

3.1 Section Infos générales

Cette page constitue la première étape de saisie de données de la phase de Précalibrage dans Easy_ACC (index de page 1). Son objectif est de capturer les paramètres fondamentaux du projet — nom, localisation géographique, contraintes de distance et type d'opération, qui sont référencés par toutes les pages suivantes.

L'implémentation se trouve dans pages/precalibrage/general.py et l'accès est géré par le routeur dans app.py.

3.2 Contexte de l'Application



La navigation est assurée par la barre latérale (sidebar_precalibrage.py), qui gère les boutons Précédent / Suivant et déclenche une sauvegarde silencieuse à chaque changement de page via la fonction _nav_save().

Mise en Page

La page utilise une disposition en **deux colonnes** (st.columns([1, 2])) :

- col1 : 1/3Configuration du projet — tous les champs de saisie
- col2 : 2/3Localisation du projet — carte interactive

3.3 Composants de l'Interface (UI)

1. Nom de l'opération (st.text_input)

- Rôle : Définit un libellé lisible pour le projet actuel.
- Déclencheur : on_change → auto_save_general_field.
- Usage : Utilisé comme argument name dans les appels save_project() et affiché sur les autres pages via l'état de session.

2. Code postal (st.text_input)

- Rôle : Sert de base à la recherche géographique.
- Fonctionnement : Dès qu'il est valide, la colonne col2 effectue un géocodage en temps réel et centre la carte sur les coordonnées résolues.

3. Année de référence (st.number_input)

- Rôle : Enregistre l'année civile utilisée pour l'alignement des courbes de charge dans la phase Bilan.
- Valeur par défaut : Année en cours.

4. Distance contrainte (st.selectbox)

- Rôle : Définit le périmètre géographique maximal de l'opération de partage d'énergie.
- Options : "2 km", "10 km", "20 km" (définies dans utils/helpers.py).

5. Type d'opération (st.selectbox)

- Rôle : Classifie le type d'opération ACC (actuellement limitée à "Ouverte").

3.4 Carte Interactive et Géocodage

La carte est générée avec Folium et intégrée via streamlit_folium.

- Service de Géocodage : Utilise l'API Nominatim (OpenStreetMap) via la fonction get_coordinates_from_postal_code.
- Cache : Les résultats sont mis en cache pendant 1 heure (ttl=3600) pour optimiser les performances.
- Affichage :
    - Fond de carte : OpenStreetMap.
    - Zoom initial : 12.
    - Marqueur : Bleu, avec une fenêtre contextuelle affichant "Code postal - Ville".

3.5 Mécanisme de Sauvegarde Automatique

Chaque widget de la colonne col1 est lié à la fonction auto_save_general_field(state_key, widget_key) :

- Synchronisation : La valeur du widget (clé interne _nom_du_champ) est copiée vers la clé canonique de l'état de session (nom_du_champ).
- Persistance : Si un project_id est présent, l'état complet est sérialisé et enregistré dans la base de données SQLite via save_project().
- **[!IMPORTANT]** La sauvegarde automatique ne s'active que si un projet est activement chargé (project_id non nul). Sinon, les modifications restent temporaires dans la session en cours.

3.6 Résumé des Clés de l'État de Session

Clé	Type	Description
project_name	str	Libellé du projet
postal_code	str	Code postal à 5 chiffres
reference_year	int	Année de référence pour les calculs
distance_constraint	str	Périmètre géographique (ex: "2 km")
operation_type	str	Classification de l'opération
project_id	int	Identifiant unique en base de données

4. Module d’insertion des points d’injection

4.1 Introduction et Objectifs

Le module permet aux utilisateurs de déclarer et configurer tous les sites de production énergétique participants à une opération d'autoconsommation collective.

Les objectifs principaux de ce module sont de centraliser l'information sur les producteurs d'énergie, qu'ils soient solaires ou éoliens, de collecter et valider les courbes de production, qu'elles soient historiques ou modélisées, de gérer les informations techniques et administratives de chaque site, et enfin de préparer les données nécessaires au calcul de répartition énergétique.

4.2 Architecture et Organisation

Le module est structuré autour de deux onglets principaux accessibles depuis la page Production du précalibrage.

- « Gestion des points » : l’interface principale permettant de créer, modifier et administrer les points d’injection. Il présente un tableau récapitulatif des installations ainsi que des formulaires d’ajout et d’édition.

- « Vérification des contraintes » : dédié à la validation des distances. Il permet de vérifier que tous les producteurs respectent les contraintes de distance et disposent de données complètes pour les calculs énergétiques.

Le fichier est situé dans *pages/precalibrage/production.py*. Il interagit avec plusieurs services techniques :

- le géocodage d’adresses (*services/geolocation.py*),
- la modélisation photovoltaïque (*services/pvgis.py)*,
- le pipeline de traitement des courbes (*services/curve_processing/*)
- la persistance en base de données (*services/database.py*).

4.3 Informations Collectées par Point d'Injection

Les données d’identification comprennent le nom du producteur, le type d’énergie, le segment tarifaire et le point de livraison physique.

Les caractéristiques techniques incluent la puissance installée, ainsi que, dans le cas du solaire, l’inclinaison et l’orientation des panneaux.

La localisation repose sur une adresse postale et des coordonnées GPS pouvant être obtenues automatiquement ou saisies manuellement.

L’ensemble des points est stocké dans st.session_state["points_injection"].

4.4 Modes d'Acquisition des Courbes de Production

La première méthode d'acquisition repose sur l'import de fichiers via *st.file_uploader*. L'utilisateur peut téléverser des fichiers Excel ou CSV contenant des données historiques. Ces fichiers sont lus avec *pd.read_csv()* ou *pd.read_excel()*

Puis validés et transformés à l'aide de la fonction **process_curve()** afin de garantir leur cohérence temporelle.

La seconde méthode repose sur la modélisation via PVGIS, implémentée dans la fonction *compute_pv_curve()*. Le système interroge l'API correspondante à l'aide de la bibliothèque pvlib en utilisant les coordonnées GPS, l'inclinaison et l'orientation des panneaux, les pertes estimées et la puissance installée.

4.5 Traitement et Validation des Données

Les courbes de production sont traitées via un pipeline automatisé dans services/curve_processing/, garantissant la qualité, la cohérence temporelle et la traçabilité des données.

Point d'Entrée : *process_curve()*

La fonction process_curve(file_or_df) (dans integration.py) constitue le point d'entrée du pipeline et orchestre quatre étapes séquentielles.

La première étape repose sur r*ead_curve()*, qui assure la lecture et la détection automatique du format. Le système supporte plusieurs formats (EMS, SGE, Archelios, CSV, Excel) et retourne un DataFrame standardisé accompagné de métadonnées décrivant la source.

La deuxième étape, *resample_curve()*, normalise les données sur un pas horaire (PT60M). Elle garantit l'homogénéité des courbes en renommant la colonne principale en value et en convertissant l'index en DatetimeIndex si nécessaire.

La troisième étape, *impute_by_week_shift()*, applique une stratégie d'imputation basée sur un décalage hebdomadaire. Pour chaque valeur manquante, le système recherche des données similaires sur les semaines voisines (paramètre max_weeks, par défaut 4). Cette étape ajoute des colonnes de traçabilité (_imputed, _impute_source) et génère un rapport statistique.

Enfin, *validate_curve()* réalise une validation globale en vérifiant la continuité temporelle, la cohérence des valeurs et en calculant des indicateurs de qualité. Les anomalies éventuelles sont remontées sous forme de warnings.

Le pipeline retourne une structure contenant un indicateur de succès, le DataFrame final horaire, les métadonnées de source, les résultats de validation, un rapport d'imputation détaillé ainsi qu'une liste d'erreurs éventuelles.

4.6 Géolocalisation et Contraintes de Distance

La géolocalisation automatique repose sur la fonction *get_coordinates_from_address()*, qui convertit une adresse en coordonnées GPS stockées dans la structure du point. Une saisie manuelle reste possible pour affiner ces valeurs.

Le calcul du périmètre est réalisé par la fonction *compute_optimal_center_and_radius()*, qui détermine un centre géographique et un rayon minimal englobant l'ensemble des acteurs.

Une visualisation cartographique est générée à l'aide de la bibliothèque *Folium*.

4.7 Agrégation et Préparation pour les Calculs

Seuls les points actifs disposant de données valides sont retenus. Les courbes sont alignées sur une même année de référence afin de permettre leur comparaison. Alignment.py (*align_curve_to_reference_year(Dataframe, year)*), permet d'aligner toutes les courbes sur une année de référence, de gérer les années bissextiles et de détecter les éventuelles heures manquantes.

Ce tableau est stocké en session pour être utilisé par les modules de calcul sans nécessiter de recalcul permanent.

4.8 Intégration avec le Système ACI

Le module joue un rôle clé dans la gestion des contrats d'autoconsommation individuelle.

Chaque point d'injection est identifié par un point de livraison unique servant de référence administrative. Lorsqu'un consommateur sélectionne un producteur partenaire, ce lien est enregistré de manière cohérente dans le système.

Lors des calculs, la production est prioritairement allouée aux consommateurs liés par un contrat ACI avant toute autre distribution.

Cette relation est conservée tout au long du processus et apparaît dans les rapports énergétiques générés.

4.9 Persistance et Sauvegarde des Données

Le système assure une sauvegarde continue et sécurisée des données.

Chaque action utilisateur déclenche une sauvegarde automatique dans une base SQLite locale. Les données sont organisées par projet avec un identifiant unique, et les courbes sont stockées séparément sous forme de datasets JSON.

Un historique des modifications est conservé afin de garantir la traçabilité. Les projets peuvent être exportés et réimportés, permettant de reprendre le travail à tout moment.

Des contrôles d'intégrité sont effectués lors du chargement pour garantir la cohérence et la validité des données.

5. Module d'insertion des points de soutirage

5.1 Introduction et Objectifs

Le module permet aux utilisateurs de déclarer et configurer tous les sites de consommation énergétique participants à une opération d'autoconsommation collective.

Les objectifs principaux de ce module sont de centraliser l'information sur les consommateurs d'énergie, de collecter et valider les courbes de consommation historiques, de gérer les informations administratives et de localisation de chaque site, et enfin de préparer les données nécessaires au calcul de répartition énergétique.

5.2 Architecture et Organisation

Le module est structuré autour de deux onglets principaux accessibles depuis la page Consommation du précalibrage.

– « Gestion des points » : l'interface principale permettant de créer, modifier et administrer les points de soutirage. Il présente un tableau récapitulatif des consommateurs ainsi que des formulaires d'ajout et d'édition.

– « Vérification des contraintes » : dédié à la validation des distances et à l'aperçu des données retenues. Il permet de vérifier que tous les consommateurs respectent les contraintes de distance, disposent de données complètes pour les calculs énergétiques, et affiche le DataFrame croisé des courbes alignées sur l'année de référence.

Le fichier est situé dans pages/precalibrage/consommation.py. Il interagit avec plusieurs services techniques :

– le géocodage d'adresses (services/geolocation.py),

– le pipeline de traitement des courbes (services/curve_processing/),

– l'alignement calendaire (services/curve_processing/alignment.py),

– la persistance en base de données (services/database.py).

5.3 Informations Collectées par Point de Soutirage

Les données d'identification comprennent le nom du consommateur, le segment tarifaire et le point de livraison (PDL/PRM).

La localisation repose sur une adresse postale et des coordonnées GPS pouvant être obtenues automatiquement via le géocodeur ou saisies manuellement.

Les informations contractuelles incluent la déclaration d'un contrat d'autoconsommation individuelle (ACI), qui associe le consommateur à un producteur partenaire spécifique.

La courbe de consommation historique est stockée sous la clé courbe_consommation dans la structure du point, sous la forme d'un dictionnaire contenant le DataFrame traité (clé "df") et les métadonnées de source.

L'ensemble des points est stocké dans st.session_state["points_soutirage"].

5.4 Mode d'Acquisition des Courbes de Consommation

Contrairement aux points d'injection, les courbes de consommation ne peuvent pas être modélisées : elles reposent exclusivement sur des données historiques mesurées.

La méthode d'acquisition repose sur l'import de fichiers via st.file_uploader. L'utilisateur peut téléverser des fichiers CSV, XLS ou XLSX exportés depuis son gestionnaire de réseau ou système de supervision.

Ces fichiers sont lus avec pd.read_csv() ou pd.read_excel(), puis validés et transformés à l'aide de la fonction process_curve() afin de garantir leur cohérence temporelle.

Les formats de fichier supportés sont les suivants :

– SGE (Gestionnaire de Réseau de Distribution) : fichier CSV avec colonnes « Horodate » et « Valeur », données en heure locale française, pas de 30 minutes.

– EMS (Energy Management System) : fichier CSV avec colonnes nom, date, ID de variable et valeur. Les timestamps sont au format ISO 8601 UTC (suffixe Z) et sont automatiquement convertis en heure locale Paris (Europe/Paris) lors de l'import.

– ALEX : fichier CSV à deux colonnes (datetime, W), timestamps UTC (suffixe Z) convertis en heure locale Paris. Le format est identifié par la présence du motif ISO T...Z et l'absence de colonne supplémentaire.

– Archelios : fichier CSV avec timestamps en heure locale française au format jj/mm/aa HH:MM.

La détection du format est automatique et s'effectue dans read_curve() (services/curve_processing/io.py).

Pour les formats EMS et ALEX dont les timestamps sont en UTC, la conversion vers l'heure locale Paris est appliquée avant tout traitement afin de garantir la cohérence calendaire des données.

5.5 Traitement et Validation des Données

Les courbes de consommation sont traitées via le même pipeline automatisé que les courbes de production, dans services/curve_processing/, garantissant la qualité, la cohérence temporelle et la traçabilité des données.

Point d'Entrée : process_curve()

La fonction process_curve(file_or_df) (dans integration.py) constitue le point d'entrée du pipeline et orchestre quatre étapes séquentielles.

La première étape repose sur read_curve(), qui assure la lecture et la détection automatique du format. Le système supporte les formats EMS, SGE, ALEX, Archelios, CSV et Excel, et retourne un DataFrame standardisé accompagné de métadonnées décrivant la source.

La deuxième étape, resample_curve(), normalise les données sur un pas horaire (PT60M). Elle garantit l'homogénéité des courbes en renommant la colonne principale en value et en convertissant l'index en DatetimeIndex si nécessaire. Les fichiers SGE en pas de 30 minutes sont ainsi agrégés par somme sur chaque heure.

La troisième étape, impute_by_week_shift(), applique une stratégie d'imputation basée sur un décalage hebdomadaire. Pour chaque valeur manquante, le système recherche des données similaires sur les semaines voisines (paramètre max_weeks, par défaut 4). Cette étape ajoute des colonnes de traçabilité (_imputed, _impute_source) et génère un rapport statistique.

Enfin, validate_curve() réalise une validation globale en vérifiant la continuité temporelle, la cohérence des valeurs et en calculant des indicateurs de qualité. Les anomalies éventuelles sont remontées sous forme de warnings.

Le pipeline retourne une structure contenant un indicateur de succès, le DataFrame final horaire, les métadonnées de source, les résultats de validation, un rapport d'imputation détaillé ainsi qu'une liste d'erreurs éventuelles.

5.6 Géolocalisation et Contraintes de Distance

La géolocalisation automatique repose sur la fonction get_coordinates_from_address(), qui convertit une adresse en coordonnées GPS stockées dans la structure du point. Une saisie manuelle reste possible pour affiner ces valeurs.

Le calcul du périmètre est réalisé par la fonction show_map_with_radius(), qui détermine un centre géographique optimal et génère un cercle de rayon configurable englobant l'ensemble des acteurs.

Une visualisation cartographique est générée à l'aide de la bibliothèque Folium. Les points respectant la contrainte de distance apparaissent en vert, les points hors périmètre en rouge.

5.7 Agrégation et Préparation pour les Calculs

Seuls les points actifs disposant de données valides sont retenus. Une courbe est considérée valide si elle contient au moins 8 736 heures consécutives (364 jours) sans interruption.

Pour chaque courbe valide, l'année dominante est identifiée et utilisée comme année source. La courbe est alors réindexée sur l'année source complète puis alignée sur l'année de référence du projet via align_curve_to_reference_year() (services/curve_processing/alignment.py).

Lorsque plusieurs consommateurs sont présents, find_max_common_calendar_range() détermine la plage calendaire commune maximale entre toutes les courbes valides avant l'alignement.

Le DataFrame croisé résultant est stocké dans st.session_state["df_conso"] pour être utilisé par les modules de calcul.

5.8 Intégration avec le Système ACI

Le module gère les contrats d'autoconsommation individuelle (ACI) du côté consommateur.

Chaque point de soutirage peut déclarer un contrat ACI avec un producteur partenaire spécifique, sélectionné parmi les points d'injection enregistrés dans le projet. Ce lien est stocké dans la clé aci_partenaire de la structure du point.

Lors des calculs, la production du producteur partenaire ACI est prioritairement allouée à ce consommateur avant toute autre distribution selon la clé de répartition.

5.9 Persistance et Sauvegarde des Données

Le système assure une sauvegarde continue et sécurisée des données.

Chaque action utilisateur déclenche une sauvegarde automatique dans la base SQLite locale via save_project(). Les données sont organisées par projet avec un identifiant unique, et les courbes de consommation sont stockées séparément sous forme de datasets JSON dans la table Dataset.

6. Module de configuration des clés de répartition

6.1 Architecture générale du module

Le module de configuration des clés de répartition est implémenté dans une fonction principale *render()* utilisée dans l'interface développée avec la bibliothèque Streamlit.

Ce module permet à l'utilisateur de définir les règles selon lesquelles l'énergie produite est répartie entre les différents consommateurs de l'opération d'autoconsommation collective.

Le module remplit quatre fonctions principales :

1. récupération de la liste des consommateurs depuis l'état global de l'application
2. configuration du mode de répartition énergétique
3. saisie et validation des paramètres associés à chaque mode
4. sauvegarde automatique des paramètres du projet dans la base de données

Les paramètres définis dans ce module sont ensuite utilisés par le module de calcul du bilan énergétique afin de simuler la distribution de l'énergie.

6.2 Gestion de l'état de l'application

Le module repose sur le mécanisme de gestion d'état fourni par Streamlit via :

st.session_state

Cette structure permet de stocker les paramètres du projet ainsi que les valeurs saisies par l'utilisateur dans l'interface.

Plusieurs variables importantes sont utilisées :

Points de soutirage

points_soutirage

Liste des consommateurs définis dans le projet.

Mode de répartition

repartition_mode

Détermine la méthode utilisée pour répartir l'énergie.

Pourcentages statiques

consumer_percentages

Dictionnaire associant chaque consommateur à un pourcentage de répartition pour la clé statique.

Priorités dynamiques

consumer_priorities

Dictionnaire définissant la priorité de chaque consommateur pour la clé dynamique simple.

Configuration des groupes dynamiques

consumer_group_keys

Structure stockant les règles de répartition pour chaque groupe de priorité pour la clé dynamique simple.

6.3 Mécanisme de relancement de l'application

L'interface Streamlit fonctionne selon un modèle d'exécution complet du script à chaque interaction utilisateur.

Afin de déclencher un recalcul ou une mise à jour de l'interface, le module implémente une fonction utilitaire :

safe_rerun()

Cette fonction tente plusieurs méthodes compatibles avec différentes versions de Streamlit :

1. utilisation de l'API officielle *st.experimental_rerun()*
2. levée de l'exception interne *RerunException*

Ce mécanisme garantit la compatibilité du module avec plusieurs versions de la bibliothèque.

6.4 Sauvegarde automatique des paramètres

Le module implémente un mécanisme d'enregistrement automatique des paramètres du projet.

Fonction utilisée :

auto_save_params_field()

Cette fonction réalise plusieurs opérations :

1. mise à jour de la variable correspondante dans *session_state*
2. sérialisation complète de l'état de l'application via la fonction *serialize_state()*
3. sauvegarde dans la base de données via la fonction *save_project()*

Les informations enregistrées incluent notamment :

- nom du projet
- type de configuration
- paramètres de répartition
- état complet de la session utilisateur

Cette approche permet de restaurer un projet ultérieurement avec l'ensemble de ses paramètres.

6.5 Nettoyage des consommateurs

Avant d'afficher l'interface de configuration, le module vérifie la cohérence des données.

La liste des consommateurs est reconstruite à partir de :

points_soutirage

Chaque consommateur correspond à un point de soutirage.

Les structures suivantes sont ensuite nettoyées afin d'éliminer les consommateurs supprimés :

consumer_percentages
consumer_priorities

Le nettoyage consiste à conserver uniquement les clés correspondant aux consommateurs encore présents dans le projet.

Cette étape évite les incohérences entre l'interface et les données stockées.

6.6 Modes de répartition disponibles

Le module permet de configurer trois modes de répartition :

1. clé par défaut
2. clé statique
3. clé dynamique simple

Le choix est réalisé via un composant d'interface :

st.radio()

La sélection est automatiquement sauvegardée dans :

repartition_mode

6.7 Mode de répartition par défaut

Dans ce mode, aucune configuration utilisateur n'est requise.

La répartition de l'énergie est réalisée automatiquement par l'algorithme de simulation énergétique.

Les règles appliquées sont celles implémentées dans le module de calcul du bilan énergétique.

L'interface affiche simplement une description informative :

"La clé par défaut applique des règles internes sans configuration manuelle."

Ce mode est adapté aux simulations rapides ou aux configurations simples.

6.8 Mode de répartition statique

Le mode statique permet de définir une répartition fixe de l'énergie entre les consommateurs.

Chaque consommateur se voit attribuer un pourcentage de la production disponible.

Structure utilisée

consumer_percentages

Exemple :

```
{
  "Consommateur A": 40,
  "Consommateur B": 35,
  "Consommateur C": 25
}
```

Les pourcentages sont saisis via des composants :

st.number_input()

Pour chaque consommateur, l'utilisateur peut définir une valeur comprise entre : *0 % et 100 %*

Une vérification est effectuée afin de contrôler que : *somme des pourcentages = 100 %*

Si la somme diffère de 100 %, un message d'avertissement est affiché.

Cette validation permet de garantir la cohérence de la clé de répartition.

6.9 Mode de répartition dynamique simple

Le mode dynamique simple introduit une hiérarchie entre consommateurs.

Chaque consommateur se voit attribuer une priorité entière.

Structure utilisée

consumer_priorities

Exemple :

```
{
  "Consommateur A": 1,
  "Consommateur B": 1,
  "Consommateur C": 2
}
```

Les priorités sont saisies via des champs numériques.

Valeurs possibles : *1 à 20*

Plus la valeur est faible, plus la priorité est élevée.

Les consommateurs sont ensuite regroupés par niveau de priorité.

Structure intermédiaire

groups

Exemple :

```
{
  1: ["Consommateur A", "Consommateur B"],
  2: ["Consommateur C"]
}
```

6.10 Configuration des groupes de priorité

Une fois les priorités définies, l'utilisateur peut configurer la clé de répartition pour chaque groupe.

Deux modes sont disponibles pour chaque groupe :

1. clé par défaut
2. clé statique

Cette configuration est stockée dans :

consumer_group_keys

Structure

```
{
  priorité: {
      "mode": "default" ou "static",
      "percentages": {...}
  }
}
```

Si le mode statique est choisi, des pourcentages doivent être définis pour chaque membre du groupe.

Ces pourcentages doivent également respecter la contrainte : *somme = 100 %*.

6.11 Navigation entre les étapes (clé dynamique simple)

Le module implémente une navigation en plusieurs étapes pour le mode dynamique simple.

Étape 1

Définition des priorités des consommateurs.

Étape 2

Configuration des clés de répartition pour chaque groupe.

La progression est gérée via la variable :

dynamic_step

Les transitions entre les étapes sont déclenchées par des boutons :

Suivant : définir clé par groupe
Retour
Appliquer la clé dynamique

Chaque transition déclenche un *safe_rerun()* afin de rafraîchir l'interface.

6.12 Validation et enregistrement des paramètres (clé dynamique simple)

Lorsque l'utilisateur valide la configuration dynamique simple, les paramètres sont enregistrés dans :

consumer_group_keys

Un message de confirmation est affiché :

"Clés de répartition multi-dynamiques enregistrées."

Les paramètres sont ensuite sauvegardés automatiquement dans la base de données via le mécanisme d'auto-sauvegarde.

6.13 Interaction avec le module de calcul énergétique

Les paramètres configurés dans ce module sont utilisés directement par l'algorithme de simulation énergétique.

Les structures exploitées par le module de calcul incluent :

consumer_percentages
consumer_priorities
consumer_group_keys
repartition_mode

Ces paramètres permettent au moteur de simulation de déterminer comment la production disponible doit être répartie entre les consommateurs à chaque pas de temps.

Ainsi, ce module constitue une étape essentielle de configuration du modèle énergétique avant le lancement du calcul du bilan énergétique.

7. Module de calcul et d'analyse du bilan énergétique

7.1 Architecture générale du module

Le module de bilan énergétique est implémenté dans une fonction principale render() utilisée dans l'interface développée avec la bibliothèque Streamlit.

Ce module remplit trois fonctions principales :

1. récupération et préparation des données de production et de consommation
2. calcul des flux énergétiques entre producteurs et consommateurs
3. génération des indicateurs et visualisations associés

Les opérations de calcul reposent principalement sur les structures de données de la bibliothèque pandas pour la manipulation des séries temporelles.

7.2 Données d'entrée

Les données utilisées par le module sont récupérées depuis l'état global de l'application (session_state).

Deux DataFrames principaux sont utilisés :

DataFrame de production

df_prod

Structure :

- chaque colonne correspond à un producteur
- chaque ligne correspond à un pas de temps
- les valeurs représentent une production énergétique (kWh)

DataFrame de consommation

df_conso

Structure :

- chaque colonne correspond à un consommateur
- chaque ligne correspond à un pas de temps
- les valeurs représentent la consommation énergétique (kWh)

Les valeurs manquantes sont remplacées par 0 afin d'éviter la propagation de valeurs NaN dans les calculs.

7.3 Calcul des indicateurs globaux

Deux indicateurs énergétiques globaux sont calculés :

Production totale

Somme de toutes les valeurs du DataFrame de production.

Consommation totale

Somme de toutes les valeurs du DataFrame de consommation.

Taux de couverture

Le taux de couverture mesure la part de la consommation totale pouvant être couverte par la production locale.

Formule : *taux_couverture = (production_totale / consommation_totale) × 100*

Une vérification est effectuée afin d'éviter une division par zéro lorsque la consommation totale est nulle.

7.4 Fonction principale de calcul

compute_metrics

Le calcul détaillé des flux énergétiques est réalisé par la fonction :

compute_metrics(df_prod, df_conso, mode)

Cette fonction réalise une simulation temporelle de la distribution de l'énergie entre producteurs et consommateurs.

Elle retourne :

- *surplus_prod* : surplus cumulé par producteur
- *auto_partage_total* : autoproduction partagée totale par consommateur
- *fourn_compl_total* : fourniture de complément totale par consommateur
- *auto_partage_df* : série temporelle d'autoproduction consommée par les consommateurs
- *auto_prod_df* : série temporelle d'autoproduction partagée par les producteurs

7.5 Structures de données utilisées dans l'algorithme

Plusieurs structures sont initialisées avant la simulation.

Dictionnaires cumulés

surplus_prod
auto_partage_total
fourn_compl_total

Ces dictionnaires permettent d'accumuler les résultats globaux sur l'ensemble de la période.

DataFrames temporels

auto_partage_df
auto_prod_df

Ces structures stockent les résultats à chaque pas de temps afin de pouvoir reconstruire les courbes énergétiques.

Table de correspondance ACI

consumer_ACI

Ce dictionnaire associe certains consommateurs à un producteur spécifique dans le cadre d'un contrat d'autoconsommation individuelle.

7.6 Simulation temporelle

La simulation parcourt les données pas de temps par pas de temps.

Pour chaque pas de temps *t*, plusieurs variables sont construites :

Production initiale

orig_prod

Dictionnaire contenant la production de chaque producteur au pas de temps courant.

orig_prod_total

Somme de la production totale au pas de temps courant.

Production disponible

cur_prod

Copie de la production initiale qui sera modifiée au cours de l'allocation d'énergie.

Consommation restante

cur_conso

Dictionnaire qui sera modifié au cours de l'allocation d'énergie et contenant la consommation restante de chaque consommateur.

Ces variables permettent de mettre à jour dynamiquement les quantités restantes après chaque allocation d'énergie.

7.7 Cas sans production

Si la production totale au pas de temps courant est nulle :

orig_prod_total == 0

alors toute la consommation restante est considérée comme provenant de la fourniture de complément.

Dans ce cas, aucune allocation d'autoproduction n'est effectuée.

7.8 Allocation prioritaire ACI

Avant l'application de la clé de répartition collective, le système traite les consommateurs disposant d'un contrat d'autoconsommation individuelle.

Pour chaque consommateur :

1. le producteur associé est récupéré via *consumer_ACI*
2. la production disponible du producteur est comparée à la consommation du consommateur
3. l'énergie allouée correspond au minimum entre ces deux quantités

Après l'allocation :

- la consommation restante est réduite
- la production disponible du producteur est réduite
- les totaux d'autoproduction partagée sont mis à jour

7.9 Calcul de la production restante

Après la phase ACI, la production restante est recalculée :

prod_total = sum(cur_prod.values())

Si cette production restante est nulle, toute la consommation restante devient une fourniture de complément.

7.10 Modes de répartition

Le système implémente trois modes de répartition.

1. **Mode statique**

Dans ce mode, la production restante est répartie selon des pourcentages fixes définis pour chaque consommateur.

Ces coefficients sont récupérés depuis :

consumer_percentages

Pour chaque consommateur :

allocated = prod_total × pourcentage

Deux cas sont possibles :

- si la consommation est inférieure à l'énergie allouée, l'excédent devient surplus
- sinon la consommation restante est couverte partiellement et le reste devient fourniture de complément

2. **Mode dynamique par défaut**

Dans ce mode, la production restante est répartie proportionnellement à la consommation instantanée.

La consommation totale est calculée :

conso_total = sum(cur_conso.values())

Chaque consommateur reçoit une part proportionnelle :

coef = consommation_individuelle / consommation_totale
allocated = prod_total × coef

Les mêmes règles d'allocation s'appliquent :

- satisfaction totale ou partielle de la consommation
- mise à jour de la fourniture de complément
- calcul du surplus

3. **Mode dynamique simple**

Ce mode introduit une hiérarchie de priorités entre consommateurs.

Les priorités sont stockées dans :

consumer_priorities

Les consommateurs sont regroupés par niveau de priorité dans la structure :

groups

La production restante est alors distribuée groupe par groupe.

Variable spécifique :

production_restante

Cette variable représente la production encore disponible après distribution aux groupes précédents.

Pour chaque groupe :

1. la consommation totale du groupe est calculée
2. une quantité de production est attribuée au groupe
3. la distribution interne dépend du mode du groupe :
    - distribution statique (mode statique)
    - distribution proportionnelle à la consommation (mode dynamique par défaut)

La production restante est mise à jour après chaque groupe.

7.11 Calcul du surplus

À la fin de l'allocation, un surplus total peut subsister :

surplus_total

Ce surplus est réparti entre les producteurs proportionnellement à leur production initiale :

surplus_p = surplus_total × (production_producteur / production_totale)

Le surplus cumulé par producteur est stocké dans le dictionnaire :

surplus_prod

7.12 Calcul de l'autoconsommation des producteurs

Pour chaque producteur, la quantité d'énergie réellement autoconsommée par les consommateurs est calculée comme :

autoproduction consommée = production initiale − surplus

Cette valeur est enregistrée dans le DataFrame temporel :

auto_prod_df

7.13 Calcul des indicateurs énergétiques

Une fois la simulation terminée, plusieurs indicateurs globaux sont calculés à partir des dictionnaires et DataFrames renvoyés par la fonction *compute_metrics* :

- surplus total de production
- autoproduction partagée totale
- fourniture de complément totale

Ces valeurs servent ensuite à produire les visualisations synthétiques de l'opération énergétique.

7.14 Visualisation des résultats

Les indicateurs sont affichés sous forme de graphiques à l'aide de la bibliothèque Plotly.

Les graphiques générés incluent :

- un donut comparant production et consommation totales
- un donut de répartition de la production par producteur
- un donut représentant le répartition du surplus de production par producteur
- un donut comparant la consommation couverte par l'autoproduction et par la fourniture de complément
- un donut comparant la production autoconsommée dans l'opération et le surplus de production

Le module génère également des graphiques temporels représentant :

- les courbes de consommation
- les courbes d'autoproduction partagée
- les courbes de production
- les courbes d'autoconsommation

Ces visualisations permettent d'analyser la performance énergétique de l'opération sur l'ensemble de la période étudiée.